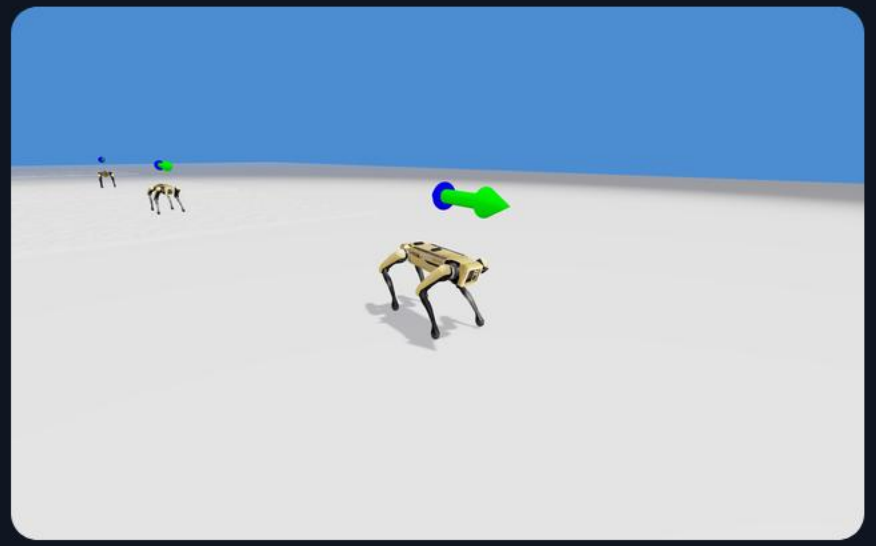
ENGINEERING WORKSHOP · ISAAC LAB 3 + NEWTON

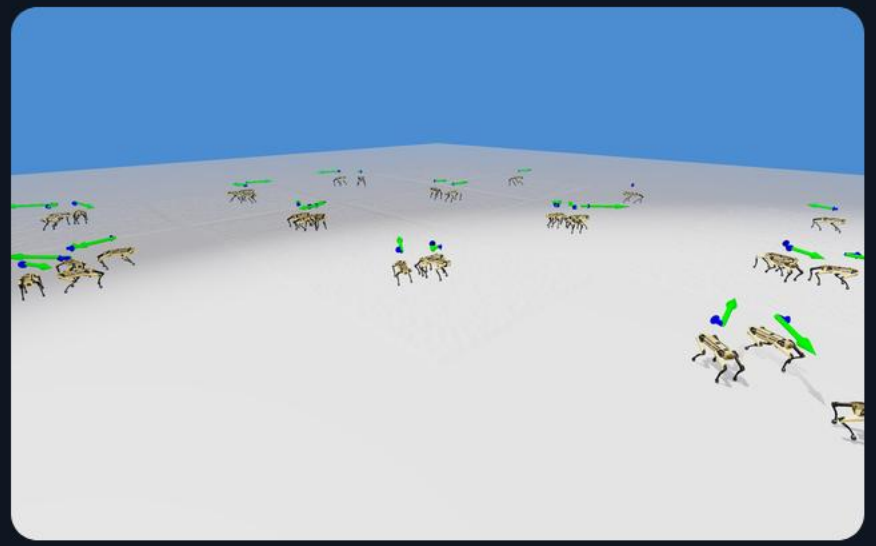
Simulation for Humanoid

Week 2 — Locomotion & Training a Quadruped

Isaac Lab 3.0 + Newton 2026 edition

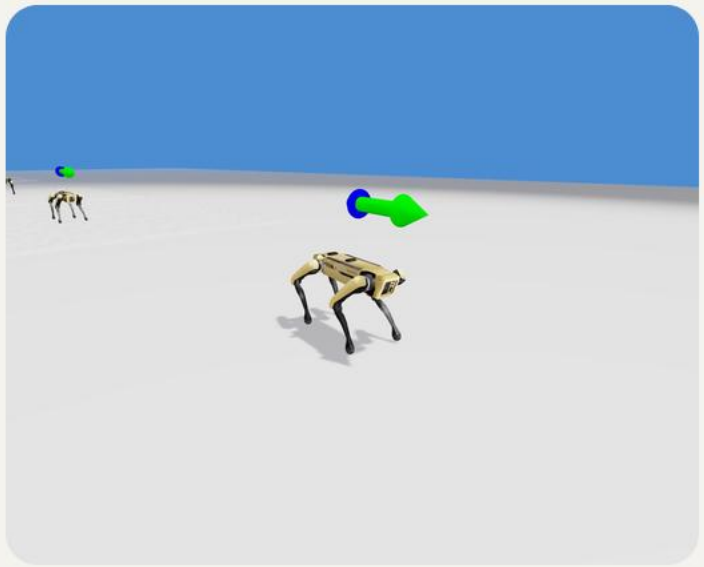
Pannatorn Bongkotamphon
Dr. Sukhum Sattaratnamai





WEEK 2

Agenda



01 Recap Week 1

02 Hands-on: เทรน Spot บน Newton

03 Locomotion (ระหว่างรอเทรน)

04 โครงสร้าง task ใน Isaac Lab 3

05 Rewards & Terrain

06 Play · Export · Exercises

RECAP FROM WEEK 1

Week 1 ได้อะไรไปแล้ว

TRAIN

```
uv run isaaclab train --rl_library rsl_rl --task Isaac-Cartpole
```

PLAY

```
uv run isaaclab play --rl_library rsl_rl --task Isaac-Cartpole \
    --checkpoint latest --viz newton
```

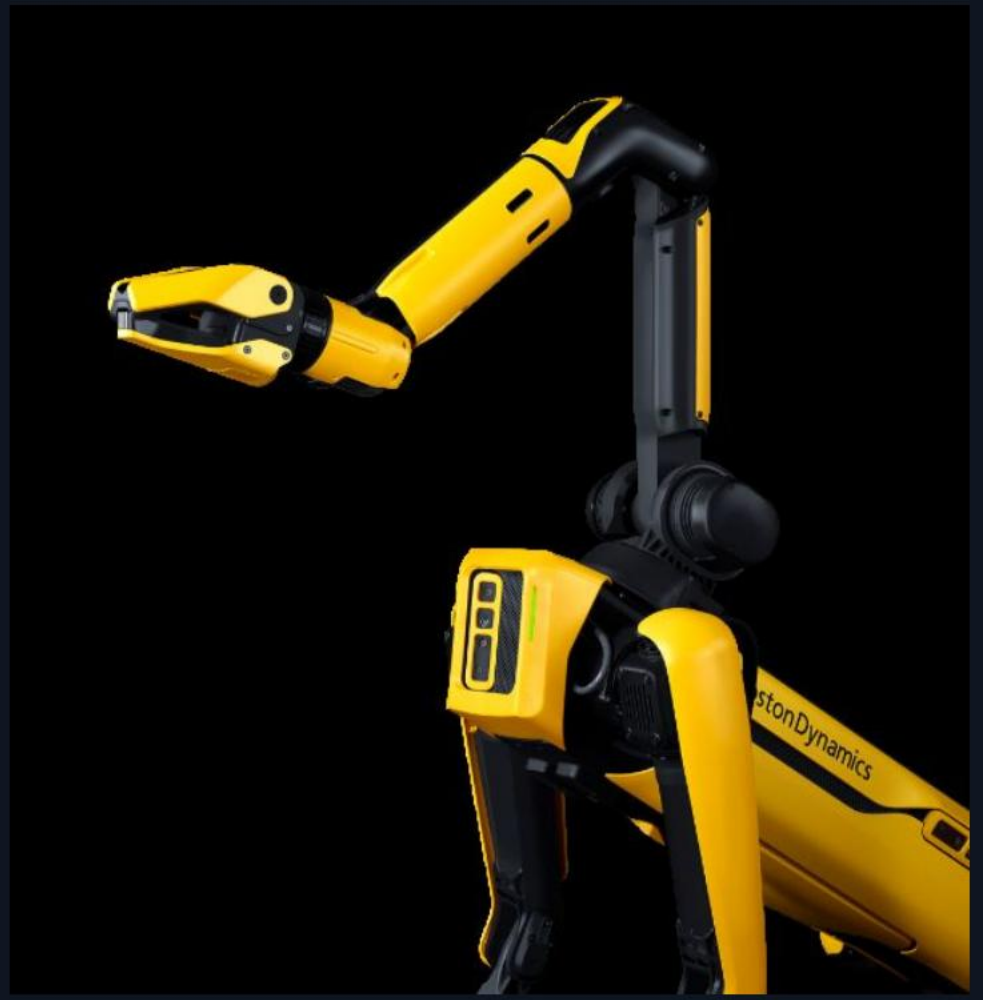
ยังรันไม่ได้? ดู **Troubleshooting** ของ Week 1

ผ่าน = viewer เปิดได้ + Cartpole ทรงตัวได้

THIS WEEK · HANDS-ON

เทรนหุ่นสี่ขา

Boston Dynamics Spot · บน Newton · ในโน้ตบุ๊กของเราเอง



ภาพ: elp-gmbh.com (Boston Dynamics Spot)

WHAT WE'LL DO THIS WEEK

จากไฟล์หุ่น → policy ที่เดินได้

1

Robot model

ไฟล์ USD / URDF
ข้อต่อ · มอเตอร์

2

RL training

reward · terrain
isaaclab train

3

Policy

model_*.pt
→ policy.pt / .onnx

4

Use it

play --viz newton
keyboard (Week 3)

Let’s start the robot training!

เริ่มเทรนก่อนเลยเพราะใช้เวลา — แล้วค่อยเรียนทฤษฎีระหว่างรอ

≈ 12 นาที

Spot 2,048 envs × 300 iterations · RTX 3060 6 GB

TRAIN QUADRUPED · STEP 0

เข้าโฟลเดอร์ Isaac Lab ก่อนเสมอ

00

```
$ cd ~/IsaacLab

# ทุกคำสั่งขึ้นต้นด้วย uv run … — uv เลือก .venv ให้เอง
```

~~conda activate env_isaaclab~~

ถ้า list_envs ขึ้น = พร้อม (Week 1)

TRAIN QUADRUPED · STEP 1

หา task ของ Spot

01

```
$ uv run isaaclab list_envs --keyword Spot
1  IsaacContrib-Velocity-Flat-Spot  ManagerBasedRLEnv
```

ชื่อเปลี่ยน!

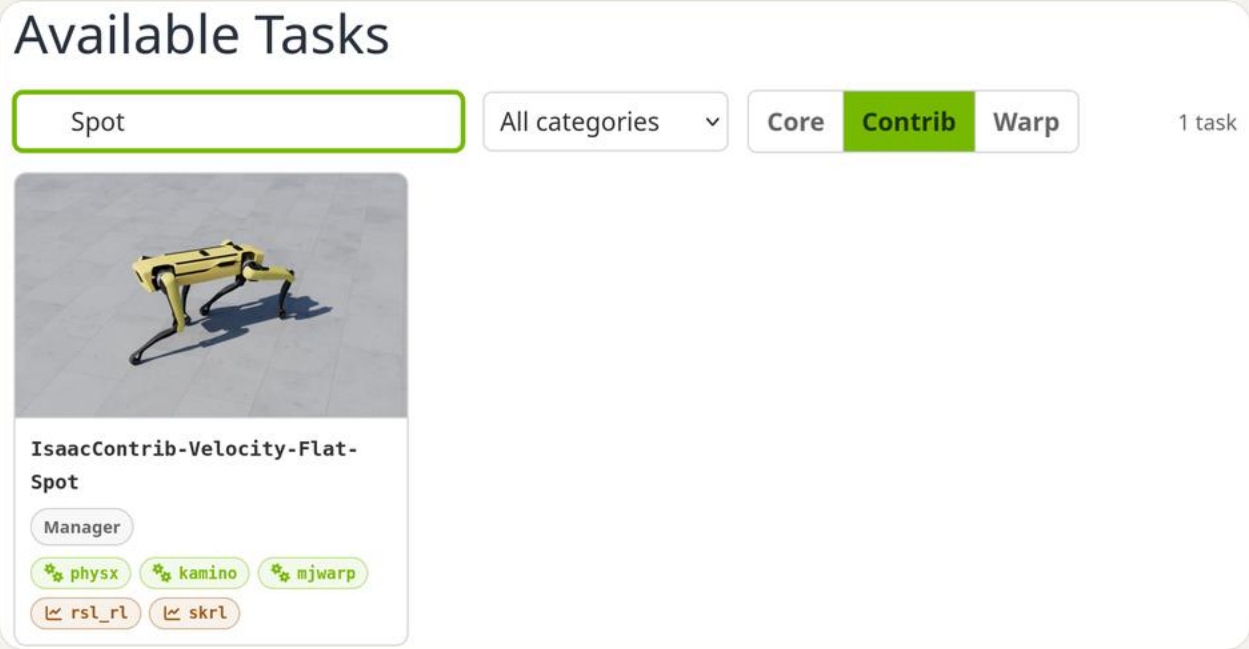
เดิม Isaac-Velocity-Flat-Spot-v0
ใหม่ **IsaacContrib-Velocity-Flat-Spot** — ไม่มี -v0

Isaac- vs IsaacContrib-

Isaac- = core tasks
IsaacContrib- = contributed tasks ที่ต่อยอดจาก core

FROM THE DOCS · AVAILABLE TASKS

Spot อยู่ในแท็บ Contrib



1 **Contrib → ค้น “Spot”**

เจอ 1 task: IsaacContrib-Velocity-Flat-Spot

2 **มี badge mjwarp**

รองรับ Newton — แต่ค่าเริ่มต้นของ task นี้คือ PhysX

3 **จึงต้องใส่ physics=newton_mjwarp**

ไม่ใส่ = ไปเรียก PhysX แล้วค้างที่ “Starting physics”

4 **Command Builder ยังไม่มี Spot**

dropdown มีแค่ core tasks → พิมพ์คำสั่งเองแบบ Step 2

เช็คแล้ว: dropdown ของ Command Builder มี 44 tasks = จำนวน Core · ภาพ: isaac-sim.github.io/IsaacLab/v3.0.0-EA/source/setup/environments.html

TRAIN QUADRUPED · STEP 2

สั่งเทรน — แยกส่วนคำสั่ง

02

```
$ uv run isaaclab train --rl_library rsl_rl \
    --task IsaacContrib-Velocity-Flat-Spot \
    physics=newton_mjwarp
```

isaaclab train

โหมดเทรน (แทน ./isaaclab.sh -p .../train.py)

--rl_library

rsl_rl · skrl · sb3 · rl_games

--task

เลือกหุ่น + สภาพแวดล้อม

physics=…

สำคัญ! Spot ค่าเริ่มต้นเป็น PhysX ไม่ใส่จะค้าง

เจอจริง: ไม่ใส่ physics= แล้วค้างที่ “Starting physics 46%” เพราะไม่ได้ลง Isaac Sim

TRAIN QUADRUPED

โปรดจำ path ผลการเทรนไว้

```
Task         IsaacContrib-Velocity-Flat-Spot
Physics      newton_mjwarp
Device       cuda:0
Environments 2048

[INFO] Logging experiment in directory:
~/IsaacLab/logs/rsl_rl/spot_flat
```

```
logs/rsl_rl/spot_flat/
└ 2026-10-08_23-39-01_workshop/
   ├ model_0.pt … model_299.pt
   ├ params/    (config ที่ใช้)
   ├ events.out.tfevents…
   └ exported/  (หลัง play)
```

ตั้งชื่อ run ได้ด้วย **--run_name workshop**

Output จริงจากเครื่องทดสอบ

TRAIN QUADRUPED · STEP 3

03

ETA นานเกินไปไหม? ปรับเอง

```
$ uv run isaaclab train --rl_library rsl_rl --task IsaacContrib-Velocity-Flat-Spot \
    --num_envs 2048 --max_iterations 300 --run_name workshop physics=newton_mjwarp
```

Argument	ความหมาย	ค่าเริ่มต้น (Spot)
`--num_envs`	จำนวนหุ่นที่เทรนพร้อมกัน	4096
`--max_iterations`	จำนวนรอบการเรียนรู้	**20000 (นานมาก!)**
`--viz newton`	เปิดหน้าต่างดู (ช้าลง)	headless

วัดจริง RTX 3060 6 GB: 2,048 envs → ~2.3 วินาที/iteration → 300 iterations = **12 นาที**

READING THE TRAINING LOG

ดูอะไรใน terminal

Mean reward

ควรค่อย ๆ ขึ้น · 1.6 (iter 19) → 132.7 (iter 299)

Mean episode length

ยิ่งยาว = ล้มน้อยลง

Episode_Reward/*

ค่าของแต่ละ reward term (ดูหัวข้อ Rewards)

Termination/body_contact

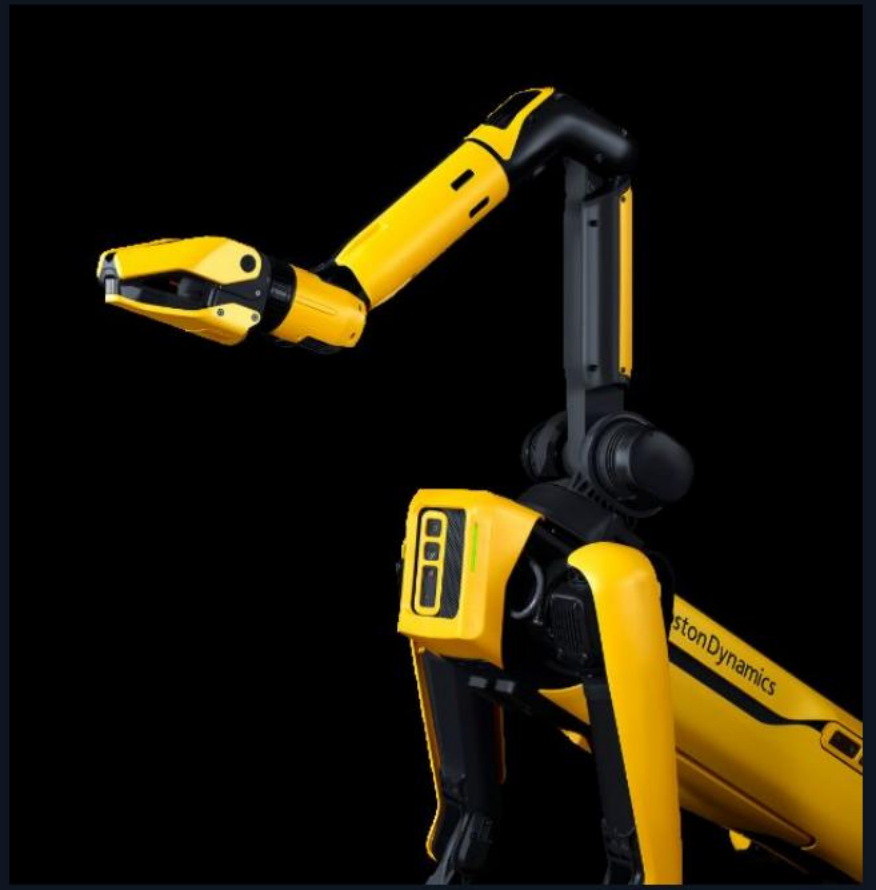
สัดส่วนที่ล้ม

```
Learning iteration 299/300
Steps per second: 21538
Mean reward: 132.69
Mean episode length: 631.20
…/air_time: 1.4032
…/base_linear_velocity: 1.3871
…/gait: 2.5482
…/action_smoothness: -1.1060
…/foot_slip: -0.3162
Termination/time_out: 0.6642
Termination/body_contact: 0.3358
Training time: 721.1 seconds
```

WHILE IT TRAINS · WHAT TO LEARN?

“Ability to stand, move”

= Locomotion



LOCOMOTION

ความสามารถในการเคลื่อนที่

นิยาม

เคลื่อนที่จากที่หนึ่งไปอีกที่ โดยใช้ **kinematics** สร้างความเร็ว

Wheeled robot

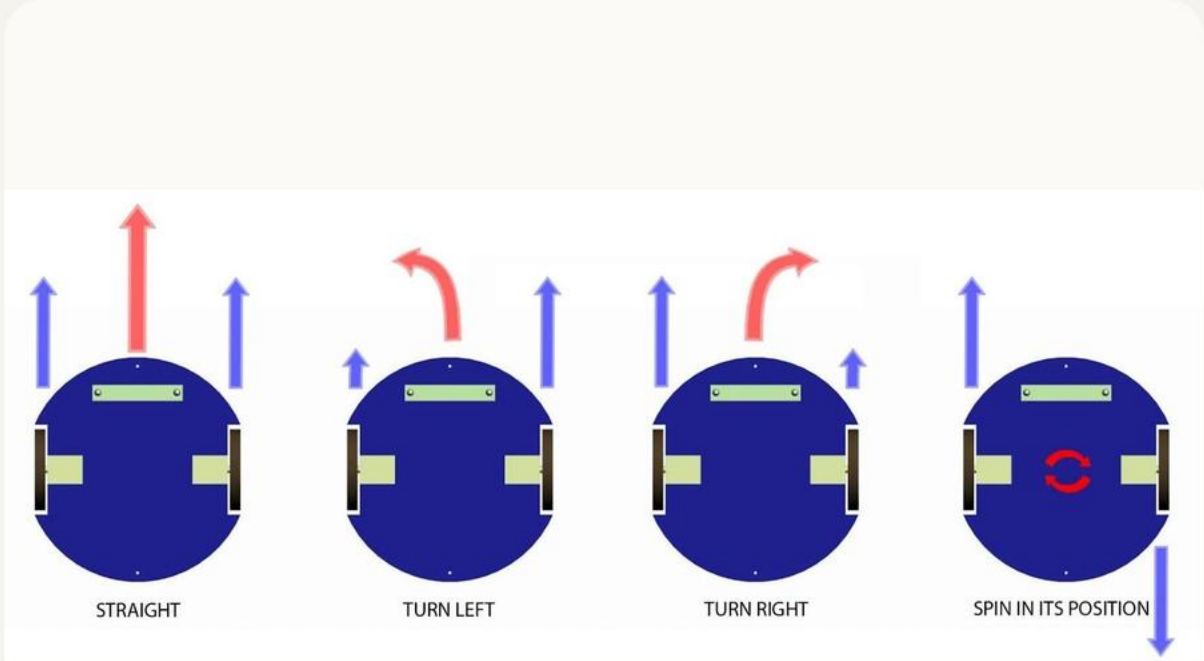
หมุนล้อ → ได้ความเร็ว

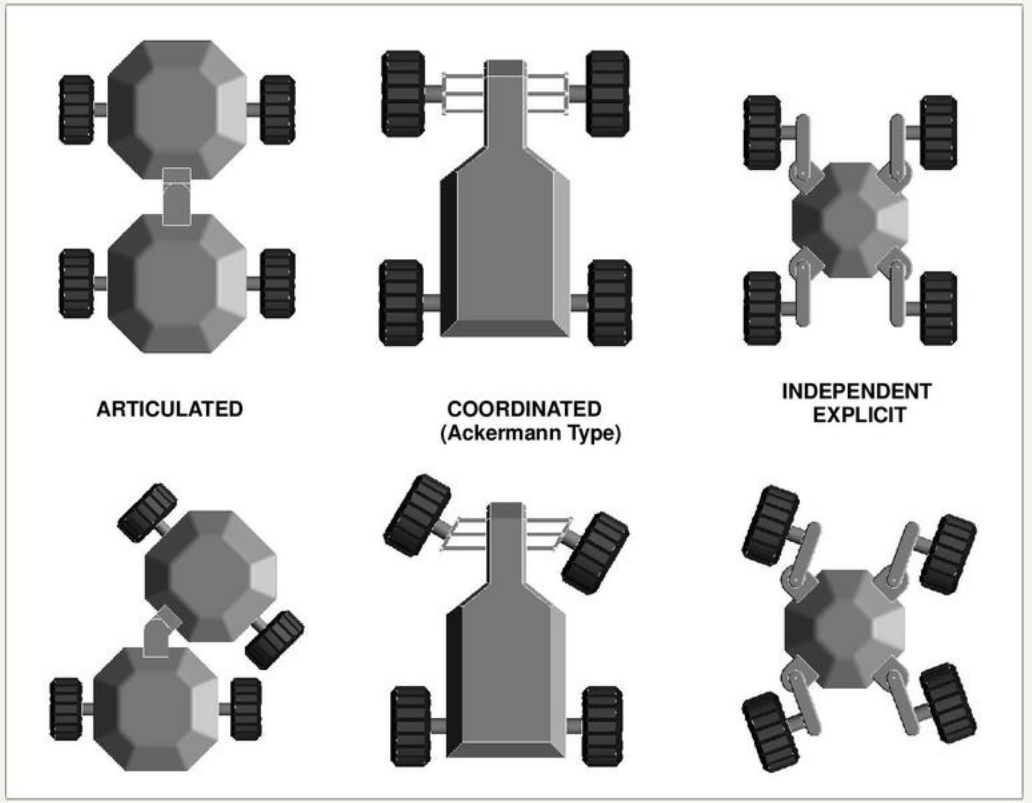
Legged robot ← workshop นี้

Standing · Walking · Running · Rolling · Jumping ...

LOCOMOTION · WHEELED ROBOT

หมุนล้อเพื่อสร้างความเร็ว

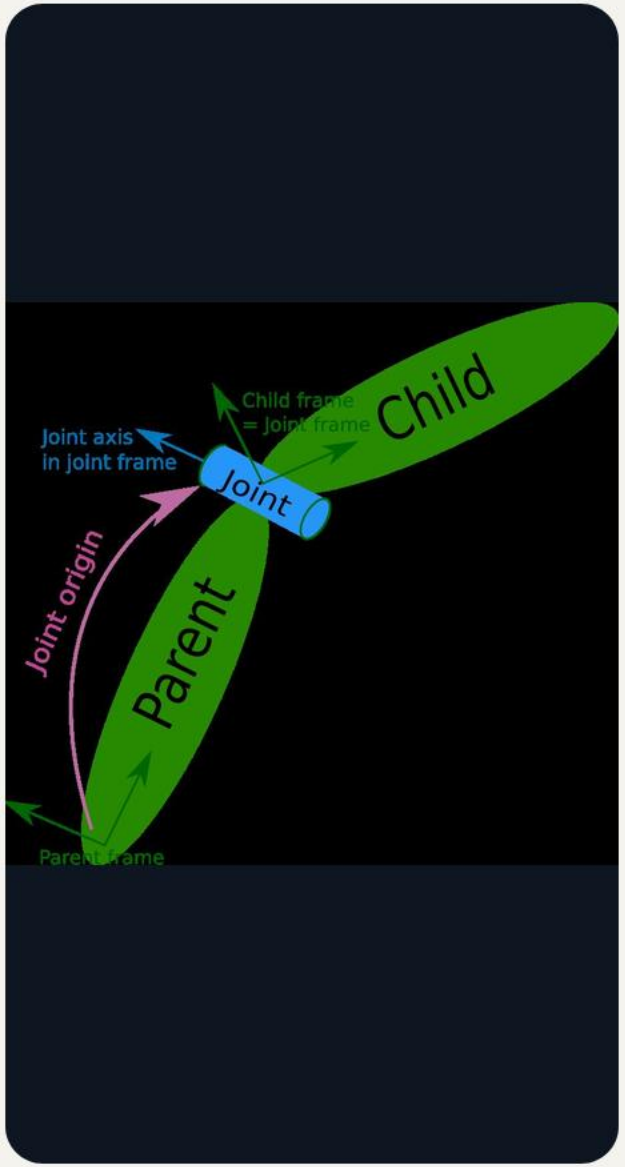


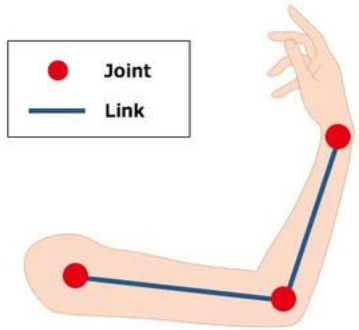


ภาพ: erc-bpgc.github.io/handbook · Whittaker & Apostolopoulos (semanticscholar)

LOCOMOTION · MANIPULATOR

แขน/ขาหุ่น = โซ่ของ Link + Joint





Link — ชิ้นส่วนแข็ง

Joint — ข้อต่อเชื่อม parent → child

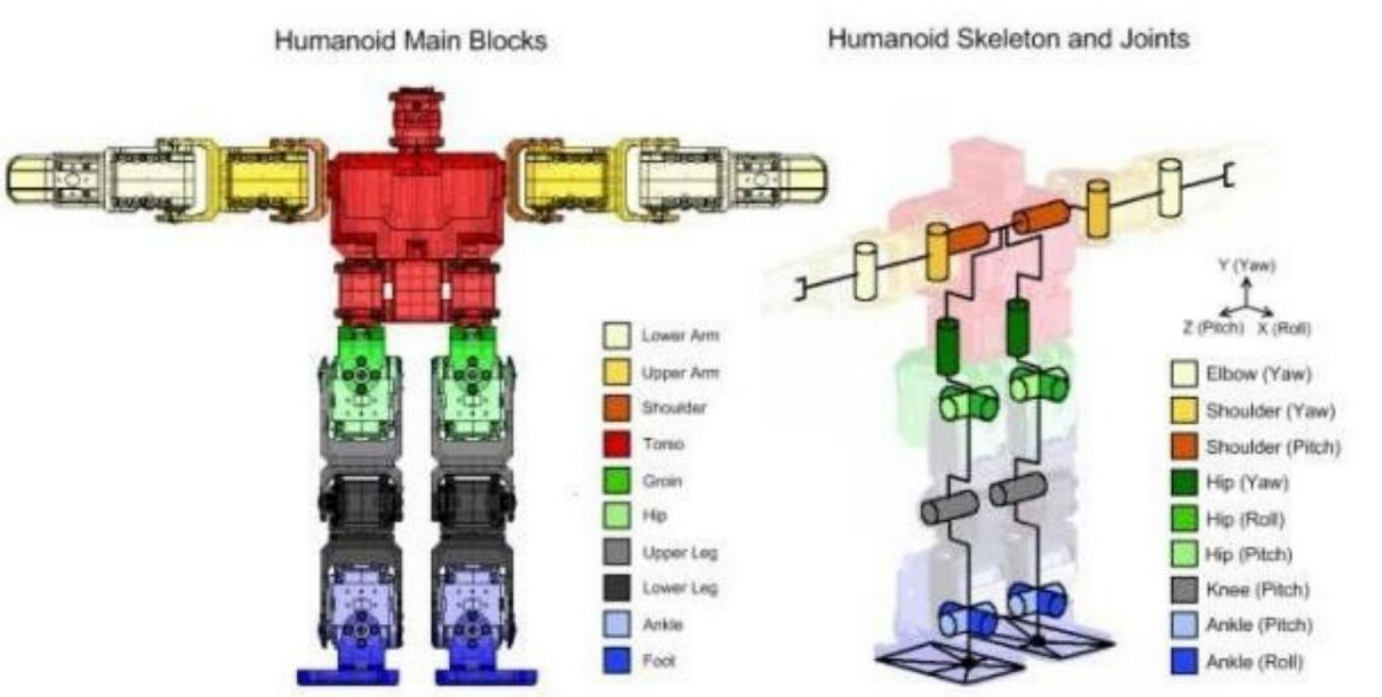
DoF — จำนวนทิศที่ขยับได้อิสระ

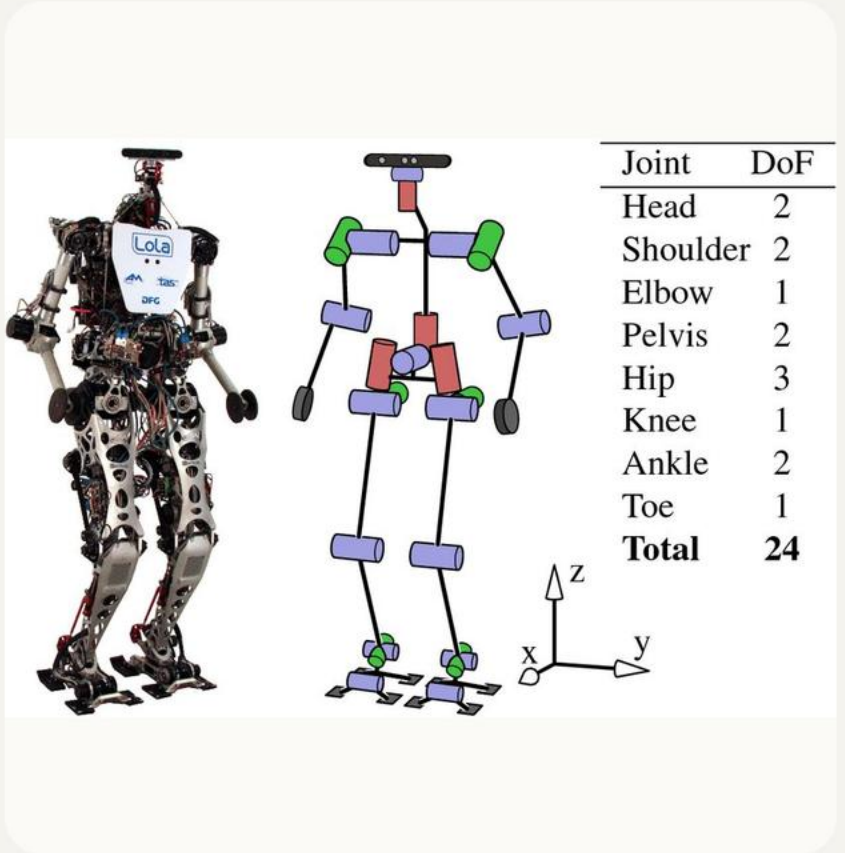
Legged — Bi-ped (2 ขา) · Quadru-ped (4 ขา)

ภาพ: wiki.ros.org/urdf/XML/joint · kawasakirobotics.com

LOCOMOTION · MANIPULATOR

หุ่นมนุษย์ = หลาย chain มาต่อกัน





Joint	DoF
Head	2
Shoulder	2
Elbow	1
Pelvis	2
Hip	3
Knee	1
Ankle	2
Toe	1
Total	**24**

ภาพ: researchgate.net (Kinematic optimization for bipedal robots · Gait planning for humanoid robots)

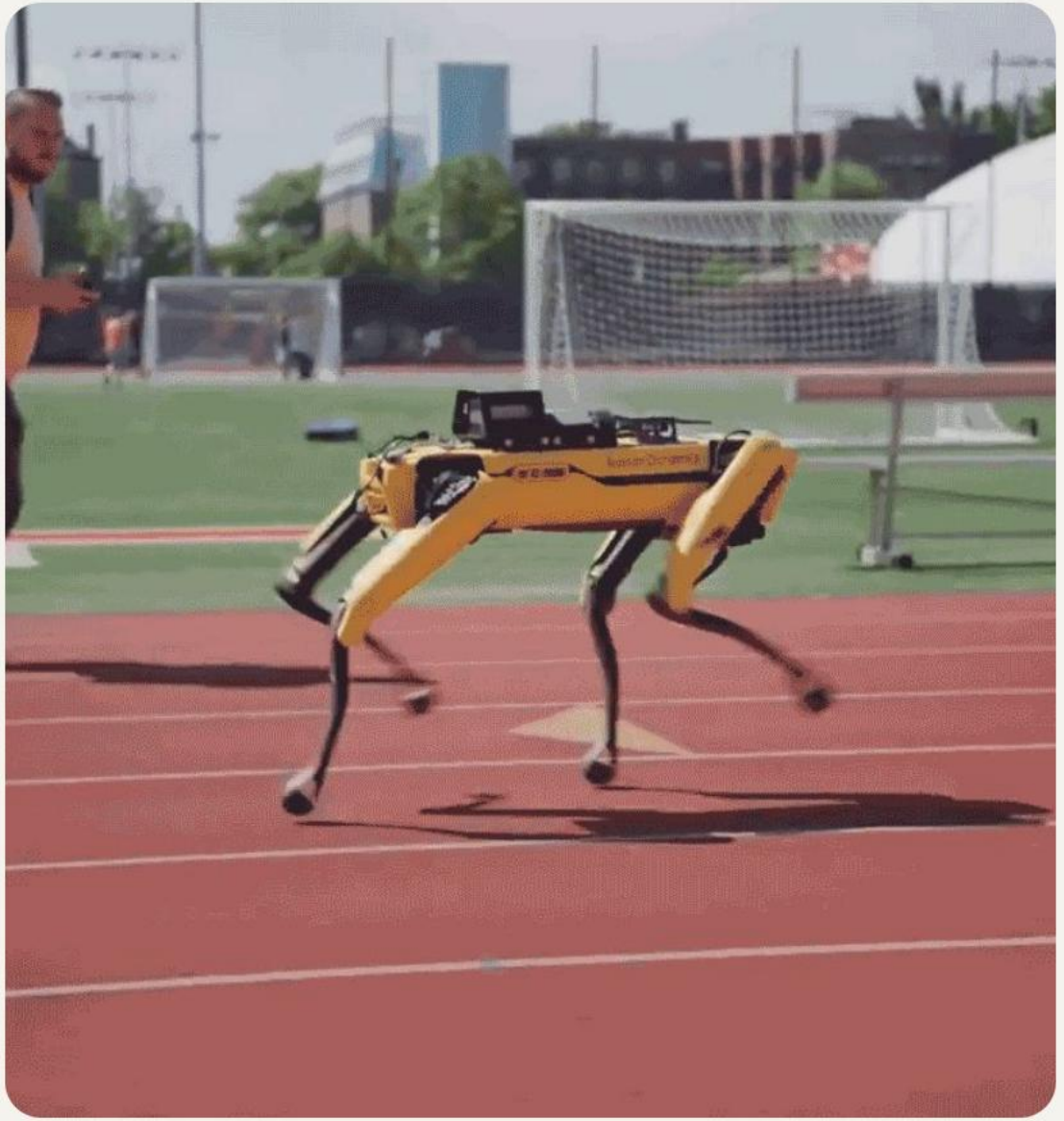
LOCOMOTION · LEGGED ROBOT

ขยับข้อต่อให้เกิด “gait” แล้วตัวถึงจะไป

1 · Gait generator — จังหวะการก้าว

2 · Footstep planner — วางเท้าตรงไหน

3 · IK / FK — แปลงเป็นมุมข้อต่อ

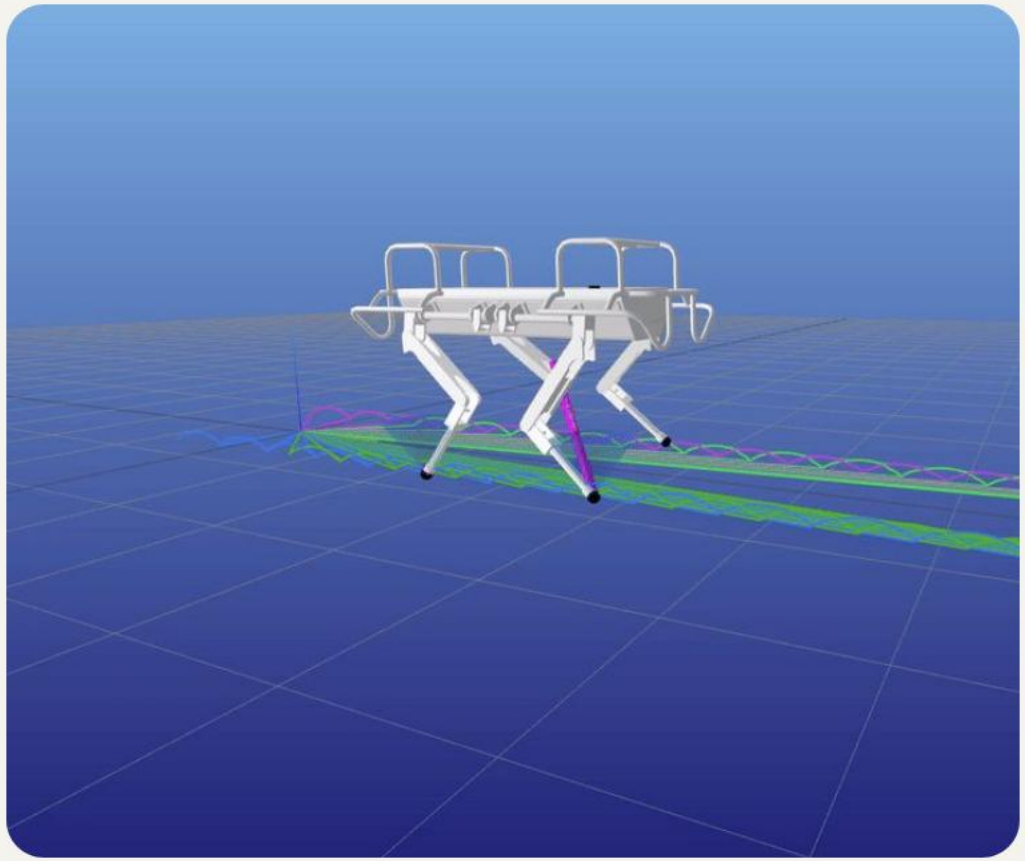


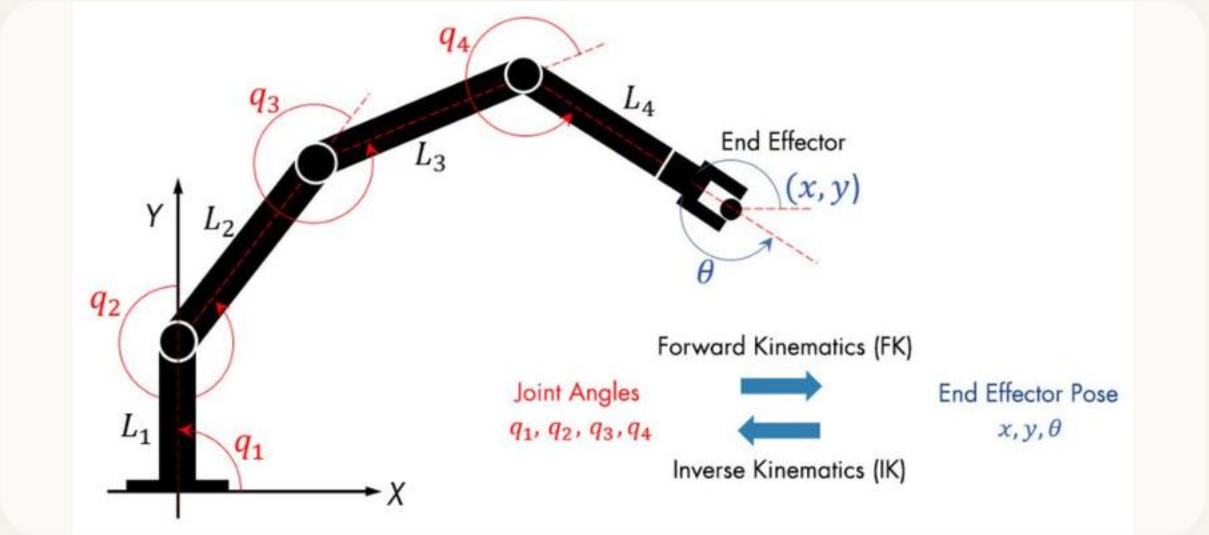
เช่น trot = ขาทแยงลงพร้อมกัน (แบบที่ Spot ใช้)

ภาพ: youtube.com/watch?v=z478hmH5OME

LOCOMOTION · LEGGED ROBOT

Forward vs Inverse Kinematics





FK

มุมข้อต่อ → ตำแหน่งเท้า

IK

ตำแหน่งเท้า → มุมข้อต่อ

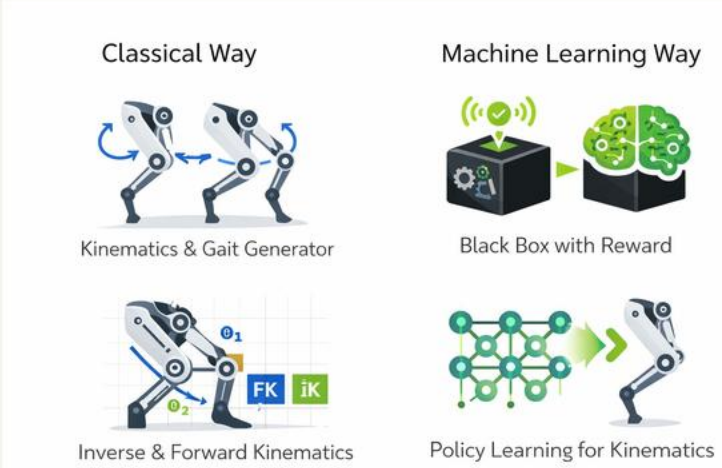
ภาพ: github.com/justagist/robot_gait_generator · sensortips.com

LOCOMOTION · TWO APPROACHES

Classical vs Machine Learning

Classical

- model kinematics + dynamics
- ออกแบบ gait เอง
- แก้ IK / FK ทุก step



ML (workshop นี้)

- black box เรียนจากประสบการณ์
- เรากำหนด **reward**
- ได้ **policy** ที่ทำ kinematics ให้เอง

OVERVIEW · REINFORCEMENT LEARNING

ลูปที่ Spot ใช้เรียนเดิน

Observation 48

ความเร็วตัว 6 · gravity 3 · คำสั่ง 3 · มุมข้อต่อ 12 · ความเร็วข้อต่อ 12 · action ก่อนหน้า 12

→

Policy (MLP)

512 → 256 → 128 · ELU
PPO · lr 1e-3 · γ 0.99

→

Action 12

มุมเป้าหมายของ 12 ข้อต่อ → PD actuator → Newton

Reward ทุก step บอกว่า “ทำดีแค่ไหน” → PPO ปรับ policy ให้คะแนนรวมสูงสุด

ตัวเลขจริงจาก contrib/velocity/config/spot/flat_env_cfg.py และ agents/rsl_rl_ppo_cfg.py

ISAAC LAB · PROJECT STRUCTURE

task ของ Spot อยู่ตรงไหน

```
isaaclab_tasks/
├ core/velocity/
│   ├ velocity_env_cfg.py
│   └ config/ g1 · h1 · go2 …
└ contrib/velocity/config/spot/
    ├ __init__.py
    ├ flat_env_cfg.py
    ├ agents/rsl_rl_ppo_cfg.py
    └ mdp/rewards.py
```

`__init__.py`
ลงทะเบียนชื่อ task

`flat_env_cfg.py`
scene · obs · actions · rewards · terrain · physics

`rsl_rl_ppo_cfg.py`
hyperparameters ของ PPO

`mdp/rewards.py`
ฟังก์ชัน reward ของ Spot

เดิม (2.x): .../manager_based/locomotion/velocity/config/spot/ — 3.0 แยกเป็น core / contrib

NEW IN 3.0 · PHYSICS PRESETS

ทำไม Spot ต้องใส่ physics=

```
# spot/flat_env_cfg.py
class PhysicsCfg(PresetCfg):
    isaacsim_physx = PhysxCfg(…)
    default = isaacsim_physx
    newton_mjwarp = NewtonCfg(
        solver_cfg=MJWarpSolverCfg(
            njmax=130, nconmax=40),
        num_substeps=2)
```

task เดียวมี physics ได้หลายแบบ เลือกด้วย override ท้ายคำสั่ง

Spot (contrib)

default = PhysX → ต้องใส่

physics=newton_mjwarp

G1 · H1 · Go2 (core)

default = newton_mjwarp อยู่แล้ว

โค้ดตัดมาจาก Isaac Lab develop (3.0.0) · docs “Multi-Backend Architecture and Presets”

QUICK TOUR · REWARDS

อะไรคือ “ดี” → reward บวก

+10	`gait`	ขาทแยงลงพื้นพร้อมกัน (trot)
+5	`air_time`	ยกเท้าตอนเดิน · วางเท้าตอนยืน
+5	`base_linear_velocity`	เดินตามความเร็วที่สั่ง (x, y)
+5	`base_angular_velocity`	หมุนตัวตามที่สั่ง (yaw)
+0.5	`foot_clearance`	ยกเท้าสูงพอ ไม่ลากเท้า

ที่มา: SpotRewardsCfg ใน contrib/velocity/config/spot/flat_env_cfg.py

QUICK TOUR · PENALTIES

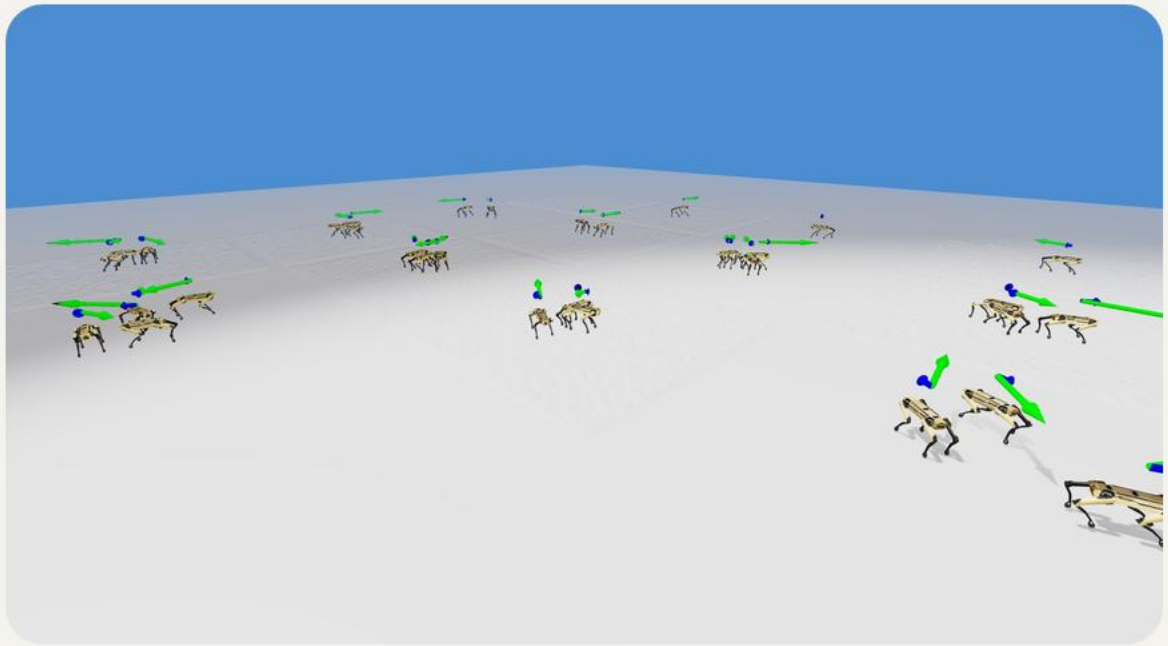
อะไรคือ “ไม่ดี” → reward ลบ

-3	base_orientation	ตัวเอียง (ไม่เอียงเดิน)
-2	base_motion	ตัวโยกขึ้นลง (ไม่ขยุ่ม)
-1	action_smoothness	คำสั่งกระตุกระหว่าง step
-1	air_time_variance	แต่ละขายกนานไม่เท่ากัน
-0.7	joint_pos	ข้อต่อห่างจากท่ายืนปกติ
-0.5	foot_slip	เท้าลื่นขณะแตะพื้น
เล็กมาก	joint_vel · torques · acc	สั่งมอเตอร์เกินจริง / เปลืองแรง

QUICK TOUR · TERRAIN

“Flat” แต่ไม่เรียบซะทีเดียว

```
COBBLESTONE_ROAD_CFG = TerrainGeneratorCfg(
    size=(8.0, 8.0),
    num_rows=9, num_cols=21,
    sub_terrains={
      "flat": MeshPlaneTerrainCfg(
          proportion=0.2),
      "random_rough": HfRandomUniform…(
          proportion=0.2,
          noise_range=(0.02, 0.05)),
    })
```



ผิวขรุขระ **2–5 ซม.** สลับพื้นเรียบ → policy ทนขึ้นเวลาเจอโลกจริง

Week 6–7 เราจะสร้าง terrain + curriculum เอง

BACK TO THE TERMINAL

เทรนเสร็จยัง?

ถ้าเห็น “Training time: ... seconds” = เสร็จ



PLAY THE POLICY · STEP 1

เลือก checkpoint — ไม่ต้องก๊อป path แล้ว

`latest`

model ล่าสุดของ run ล่าสุด · ใช้บ่อยสุด

`best`

best/final ของ library (ไม่มีก็ใช้ตัวล่าสุด)

`logs/…/model_299.pt`

ระบุไฟล์เอง

`pretrained`

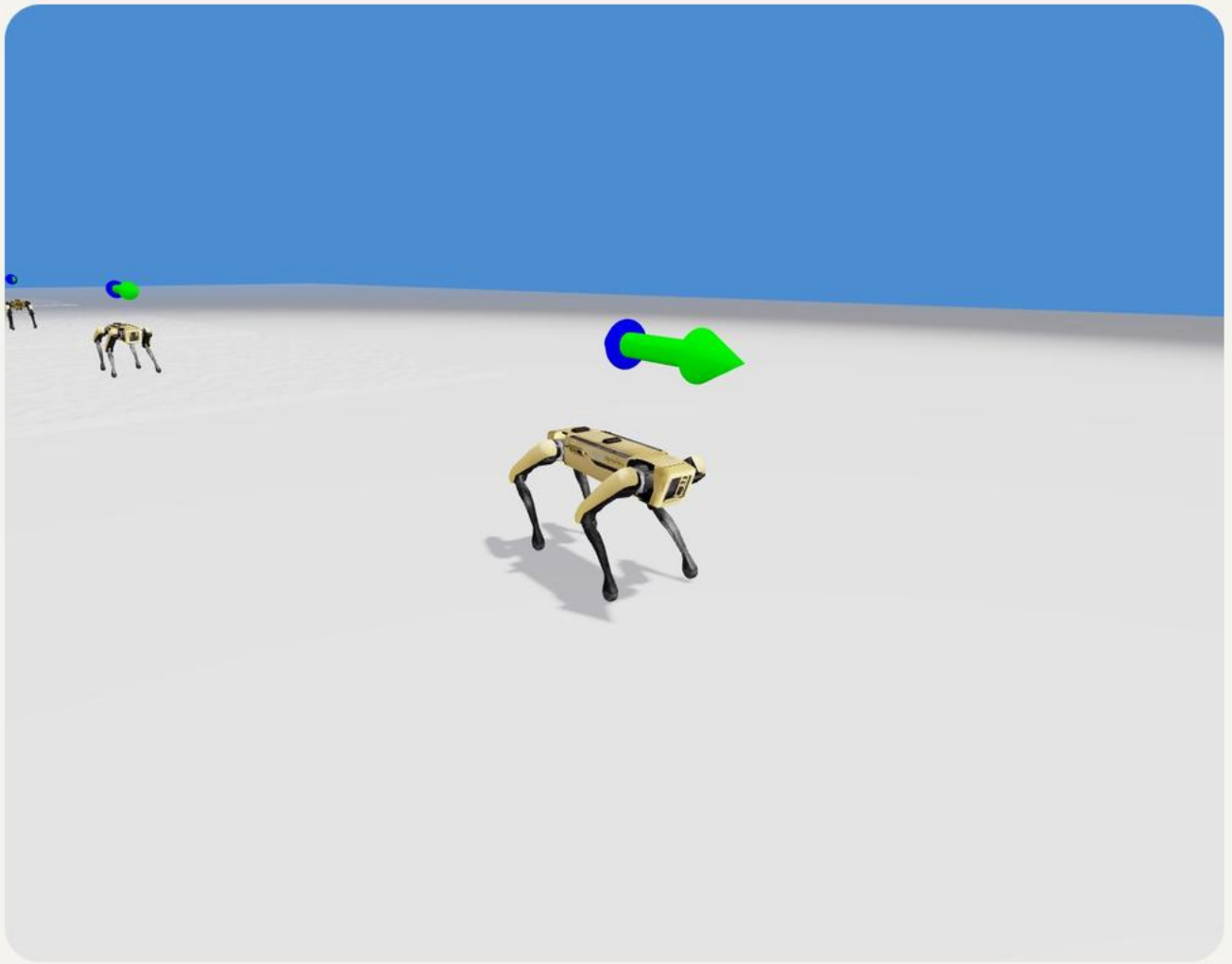
policy ที่เผยแพร่ไว้ให้ (Week 3)

ที่มา: docs “Unified Checkpoint and Iteration Arguments” · latest ทดสอบแล้ว

PLAY THE POLICY · STEP 2

ดูผลด้วย Newton viewer

```
$ uv run isaaclab play \
  --rl_library rsl_rl \
  --task \
    IsaacContrib-Velocity-Flat-Spot \
  --num_envs 16 \
  --checkpoint latest \
  --viz newton \
  physics=newton_mjwarp
```



ลูกศรเขียว = ความเร็วที่สั่ง · **ลูกศรน้ำเงิน** = ความเร็วจริง

play ไม่ต้องใช้หุ่นเยอะ — แค่อยากดูผล

PLAY THE POLICY · STEP 3 (EXTRA)

play = export policy + อัดวิดีโอได้

Export อัตโนมัติ

```
logs/rsl_rl/spot_flat/<run>/
└ exported/
   ├ policy.pt    (TorchScript)
   └ policy.onnx  (ใช้ที่อื่น/หุ่นจริง)
```

ได้ทุกครั้งที่รัน play

อัดวิดีโอ

```
uv run --extra video isaaclab play \
  … --viz newton \
  --video --video_length 300
```

→ <run>/videos/play/clip_model_299_0000.mp4

ทดสอบแล้วทั้ง 2 อย่าง · คลิปตัวอย่าง: Week02_Quadruped/media/week2_spot_play_newton.mp4

CHECKPOINT · ก่อนไปต่อ

Lab 2 ผ่านไหม?

2.1 · Training success

เทรน Spot บน Newton จนจบ — terminal ขึ้น “Training time”

2.2 · Result

play --viz newton แล้ว Spot เดินตามลูกศรเขียว (ยังไม่สวยไม่เป็นไร) · โบนัส: อัดคลิปด้วย --video

ต่อไป: Exercises — ปรับ reward แล้วดูว่าหุ่นเปลี่ยนไปยังไง





EXERCISES · WEEK 2

ทดลองกับ reward และ config

4 โจทย์ · ใช้ 1,024 envs × 150 iterations (≈ 4 นาทีต่อรอบ) เพื่อให้เทียบกันได้หลายแบบ · ตัวเลขทั้งหมดวัดจากเครื่องทดสอบ

EXERCISE 2.1 · REWARD ABLATION

ปิด reward “gait” แล้ว Spot ยังเดินได้ไหม? 2.1

```
$ uv run isaaclab train --rl_library rsl_rl --task IsaacContrib-Velocity-Flat-Spot \
    --num_envs 1024 --max_iterations 150 --run_name nogait \
    physics=newton_mjwarp env.rewards.gait.weight=0.0
```

1,024 × 150 it.	baseline	gait = 0
Mean reward	94.3	27.8
Episode length	623	426
ล้ม (body_contact)	48 %	60 %
base_linear_velocity	1.08	0.61
Training time	219 s	228 s

โจทย์

1. รัน baseline (ไม่ใส่ override) กับ nogait
2. play ทั้งสองด้วย **--checkpoint** ชี้ไปที่ run นั้น แล้วดูท่าเดิน
3. ลองปิดตัวอื่น: **foot_slip**, **base_orientation** — ตัวไหนกระทบมากสุด?

คิดต่อ: reward ที่ดูเหมือน “แค่ตกแต่งท่าเดิน” จริง ๆ ช่วยนำทางการเรียนรู้ด้วย — ตัดออกแล้ว reward หลักก็ตกตาม ไม่ใช่แค่ท่าเปลี่ยน

เฉลย · EXERCISE 2.1

gait คือ reward ที่นำทาง — อีกสองตัวคุมคุณภาพ

1. ระวัง: Mean reward ของ nogait ต่ำลงส่วนหนึ่งเพราะ**ตัด term gait ออกจากผลรวม** — ดูตัวชี้วัดอื่นแทน: ep. length 623 → 426 · ล้ม 48 → 60 % · base_linear_velocity 1.08 → 0.61
2. play run ที่ต้องการ: **--checkpoint logs/rsl_rl/spot_flat/<วันที่>_nogait/model_149.pt** (+ physics=newton_mjwarp)
3. ปิด foot_slip หรือ base_orientation: ตัวเลขแทบไม่ต่างจาก baseline ใน 150 iterations → term พวกนี้ไม่ได้ช่วยให้ “เดินได้” แต่คุมท่าทาง ต้องดูตอน play หรือเทรนนานขึ้น

1,024 × 150 it.	reward	ep. len	ล้ม
baseline	94.3	623	48 %
gait = 0	27.8	426	60 %
foot_slip = 0	97.4	623	48 %
base_orientation = 0	95.0	608	44 %

สรุป: reward แต่ละตัวมีหน้าที่ต่างกัน — ตัวนำทาง (gait, tracking) กับตัวคุมคุณภาพ (penalty) ประเมินด้วยตัวชี้วัดคนละแบบ

ก่อนทำ EXERCISE 2.2 · ENV.COMMANDS.BASE_VELOCITY.RANGES

คำสั่งความเร็วมี 3 ตัว · แต่ละตัวคือช่วง [min, max]

ชื่อ	แกน (ของลำตัวหุ่น)	ค่าเดิม Spot	ตัวแรก = min	ตัวที่สอง = max
`lin_vel_x`	เดินหน้า (+) / ถอยหลัง (–) · m/s	`[-2.0, 3.0]`	ถอยหลังได้เร็วสุด 2 m/s	เดินหน้าได้เร็วสุด 3 m/s
`lin_vel_y`	ไปทางซ้าย (+) / ขวา (–) · m/s	`[-1.5, 1.5]`	ไปทางขวาเร็วสุด 1.5 m/s	ไปทางซ้ายเร็วสุด 1.5 m/s
`ang_vel_z`	หมุนซ้าย (+) / หมุนขวา (–) · rad/s	`[-2.0, 2.0]`	หมุนขวาเร็วสุด 2 rad/s	หมุนซ้ายเร็วสุด 2 rad/s

```
# isaaclab/envs/mdp/commands/velocity_command.py · ทุก 10 s หุ่นแต่ละตัวสุ่มคำสั่งใหม่ “แยกกันทีละแกน”
vx = uniform(*lin_vel_x) · vy = uniform(*lin_vel_y) · wz = uniform(*ang_vel_z)  # 10% ของหุ่นได้ 0 ทั้งหมด (ฝึกยืน)
```

Exercise 2.2

lin_vel_x=[0.0, 1.0]

แก้แค่แกน x: ไม่มีคำสั่งถอยหลัง · เดินหน้าไม่เกิน 1 m/s — แต่ vy และ wz ยังสุ่มเหมือนเดิม หุ่นจึงยังเดินข้างและหมุนตัว

min ↑ = ตัดคำสั่งช้าออก

ถ้าแก้ตัวแรกเป็น 0.5

[0.5, 1.0] = ทุกคำสั่งต้องเดินหน้าอย่างน้อย 0.5 m/s → หุ่นไม่ได้ฝึกเดินช้า ๆ (นอกจาก 10% ที่ยืน)

ต้องแก้ 3 ค่า

เดินหน้าอย่างเดียวจริง ๆ

ต้องปิดอีก 2 แกน: lin_vel_y="[0.0,0.0]" และ ang_vel_z="[0.0,0.0]" · แต่หุ่นจะเลี้ยวไม่เป็นเลย

EXERCISE 2.2 · COMMAND RANGE

ตัดคำสั่งถอยหลังออก (แกน x)

2.2

```
$ uv run isaaclab train --rl_library rsl_rl --task IsaacContrib-Velocity-Flat-Spot \
    --run_name fwdonly physics=newton_mjwarp \
    env.commands.base_velocity.ranges.lin_vel_x="[0.0,1.0]"
# [min, max] ของแกน x · ค่าเดิม (-2.0, 3.0) → ไม่ถอยหลัง เดินหน้าไม่เกิน 1 m/s
# vy, wz ไม่ได้แก้ · list ต้องใส่ใน "..." เพราะ Hydra ไม่รู้จัก tuple
```

ผลบนเครื่องทดสอบ · RTX 3060 LAPTOP 6 GB

ทดสอบ 2 iterations ว่า override ผ่าน — ใน **logs/rsl_rl/spot_flat/<run>/params/env.yaml**:

```
ranges:
  lin_vel_x:
  - 0.0
  - 1.0
```

โจทย์

1. เทรนจริง 150+ iterations แล้ว play
2. diff **params/env.yaml** ของ base1024 กับ fwdonly — ควรต่างกันแค่ num_envs, log_dir และ lin_vel_x
3. เทียบ base_linear_velocity กับ baseline — ช่วงแคบลงเรียนเร็วขึ้นไหม? play แล้วดูว่าหุ่นยังเดินข้าง/หมุนอยู่ไหม เพราะอะไร

เฉลย · EXERCISE 2.2

ช่วงคำสั่งแคบลง = เรียนเร็วขึ้น แต่ทำได้น้อยลง

1. เทรนจริง 150 iterations: base_linear_velocity **1.44** vs baseline 1.08 · ep. len 680 vs 623 · ล้ม 40 vs 48 %
2. diff params/env.yaml ต่างกันแค่ num_envs, log_dir และ lin_vel_x — ใช้ diff แบบนี้ทุกครั้งที่จะสรุปว่าผลต่างมาจาก config ไหน
3. ใช่ — โจทย์ง่ายลง (ไม่ต้องถอยหลังหรือวิ่ง 3 m/s) policy จึง track ได้ดีขึ้นในเวลาเท่ากัน แต่ถ้าสั่งถอยหลังจะอยู่นอกช่วงที่เทรน (ดู Week 3 · Exercise 3.2) · หุ่นยังเดินข้างและหมุนได้ เพราะ lin_vel_y และ ang_vel_z ยังเป็นช่วงเดิม

lin_vel_x	lin_vel rew.	ep. len
baseline (−2.0, 3.0)	1.08	623
fwdonly (0.0, 1.0)	1.44	680

สรุป: เลือกช่วงคำสั่งตามงานจริง — กว้างเกินเปลืองเวลาเทรน แคบเกินหุ่นทำงานนอกช่วงไม่ได้

EXERCISE 2.3 · COMPARE ROBOTS

Spot vs Unitree Go2 — เทรนเท่ากัน

2.3

```
$ uv run isaaclab train --rl_library rsl_rl --task Isaac-Velocity-Flat-UnitreeGo2 \
    --num_envs 1024 --max_iterations 150 --run_name cmp1024
# core task → ใช้ Newton เป็นค่าเริ่มต้น ไม่ต้องใส่ physics=
```

1,024 × 150 it.	Spot (contrib)	Go2 (core)
Training time	219 s	68 s
Steps per second	17,427	56,430
Episode length	623	991 (เต็ม ≈ 1,000)
Mean reward	94.3	6.1

โจทย์

1. เปิด config ทั้งสอง: **contrib/.../spot/flat_env_cfg.py** vs **core/velocity/config/go2/**
2. นับจำนวน reward term และ obs ของแต่ละตัว
3. ทำไม Go2 เร็วกว่า 3 เท่า? (ขนาดหุ่น · จำนวน contact · solver)

คิดต่อ: อย่าเทียบ Mean reward ข้ามหุ่น — สูตร reward คนละชุด (Spot ~94 vs Go2 ~6) ให้เทียบ episode length, อัตราล้ม หรือ error ของความเร็วแทน

เฉลย · EXERCISE 2.3

Go2 เร็วกว่าเพราะ physics step น้อยกว่า 2.5 เท่า

1. config ที่ต้องเปิด: **contrib/velocity/config/spot/flat_env_cfg.py** และ **core/velocity/config/go2/flat_env_cfg.py** (+ velocity_env_cfg.py ที่เป็นแม่แบบ)
2. reward: Spot **14 terms** (เขียนเองทั้งชุด) · Go2 **10 terms** จากชุดมาตรฐาน (dof_pos_limits น้ำหนัก 0) · obs ทั้งคู่ **48**
3. Spot: sim.dt 0.002 × decimation 10 = **10 physics steps** ต่อ 1 action · Go2: 0.005 × 4 = **4 steps** — คุมที่ 50 Hz เท่ากัน แต่ Spot คำนวณ physics มากกว่า 2.5× (+ พื้น cobblestone)

	Spot	Go2
control	50 Hz	50 Hz
physics	500 Hz	200 Hz
reward terms	14	10
obs	48	48
steps/s	17.4k	56.4k

สรุป: ความเร็วการเทรนขึ้นกับ sim.dt / decimation / จำนวน contact มากกว่าขนาดหุ่น — ตรวจ config ก่อนเทียบเสมอ

EXERCISE 2.4 · READ THE CURVES

TensorBoard: อ่านกราฟการเทรน

2.4

```
$ uv run tensorboard --logdir logs/rsl_rl/spot_flat
TensorBoard 2.21.0 at http://localhost:6006/ (Press CTRL+C to quit)
```

โจทย์

1. เปิด **http://localhost:6006** แล้วติ๊ก run **base1024** กับ **nogait**
2. ดู **Train/mean_reward** และ **Episode_Reward/*** — term ไหนแยกกันก่อน?
3. เพิ่ม **--max_iterations 600** ให้ baseline แล้วดูว่ากราฟอิ่มตัวตรงไหน

ทดสอบแล้ว

rsl_rl เขียน **events.out.tfevents...** ไว้ในโฟลเดอร์ run ทุกครั้ง ไม่ต้องตั้งค่าเพิ่ม

skrl (Exercise 1.4) ก็ใช้ไฟล์แบบเดียวกัน → **--logdir logs** ดูทุก library พร้อมกันได้

คิดต่อ: ทุก run ขึ้นในกราฟเดียวกัน — ตั้ง **--run_name** ให้สื่อความหมาย แล้วจะเทียบง่ายมาก

เฉลย · EXERCISE 2.4

อ่านกราฟให้ครบ: reward, episode length, loss

1. เลือก run ด้วยชื่อ — **--run_name** ถูกต่อท้ายชื่อโฟลเดอร์ (วันที่_base1024, วันที่_nogait)
2. แยกกันเร็วมาก: iteration 50 episode length 137 vs 37 · base_linear_velocity 0.23 vs 0.08 — ไม่ต้องรอจบก็รู้ว่า run ไหนแย่ (หยุดก่อนได้ ประหยัดเวลา)
3. Loss/value **ไม่ได้ลดลงต่อเนื่อง** (0.17 → 0.61) เพราะ return ที่ต้องทำนายโตขึ้นตาม policy — ใน RL loss ไม่ใช่ตัววัดว่าเก่งขึ้น
4. เราดูจาก run 2,048 × 300 ของหน้า Train (ไม่ได้รัน 600): ถึง 80 % ของ reward สุดท้ายที่ iteration 103, 90 % ที่ 141 จากนั้นขึ้นช้า ๆ (122 → 133) = เริ่มอิ่มตัว

สรุป: ดูกราฟคู่กันเสมอ: reward + episode length + term ย่อย · loss ใช้ดูว่าเทรนพังไหม ไม่ใช่ดูว่าดีขึ้นไหม